Kimiya: A Chemistry-Aware Programming Language

Compiler Construction project, BCS-7G, FAST NUCES Lahore

Instructor: Dr. Faisal Aslam

Name	Roll number
Bisma Amir	23L-0659
Maryum Nauman	23L-0696
Zoha Fatima	23L-0581

1. About the project

Kimiya is a small programming language for chemistry. Formulas like `H2O` or `Ca(OH)2` can be written directly in the code, and the language checks that every element actually exists. It can balance equations, tell you if an equation you balanced yourself is wrong, and work out things like molar mass, moles, yield and the limiting reactant. Apart from that it has the usual stuff: variables, if/else, loops, functions and lists.

We did both phases in the same program, `kimiya`. Running `./kimiya program.kim` runs the program with the interpreter (Phase 1). Running `./kimiya -c program.kim` compiles it to LLVM IR instead (Phase 2), and clang then makes an executable out of the `.ll` file.

2. Installing the dependencies

We tested everything on Ubuntu. You need Bison, Flex, gcc, make and clang:

```
sudo apt update
sudo apt install build-essential bison flex clang
```

3. Compiling

Go to the project folder and run:

```
make
```

This runs Bison on `src/parser.y` and Flex on `src/lexer.l`, then compiles everything with gcc into `./kimiya`. `make clean` deletes `./kimiya` and the files generated by Bison and Flex.

4. Running

Phase 1: interpreter

```
./kimiya tests/examples/ex4_table.kim
```

Output:

```
1 g H2 -> 8.94 g water
2 g H2 -> 17.87 g water
3 g H2 -> 26.81 g water
4 g H2 -> 35.74 g water
5 g H2 -> 44.68 g water
```

Phase 2: compiler

```
./kimiya -c tests/examples/ex5_volcano.kim
clang tests/examples/ex5_volcano.ll src/runtime.c src/chem.c -Isrc -o volcano
./volcano
```

Output:

```
Runs out first: CH3COOH
CO2 produced: 1.83 g
Add more vinegar to use up all the baking soda.
```

clang might give a warning about "overriding the module target triple", you can ignore it.

Or you can do all three steps with one command:

```
make run FILE=tests/examples/ex5_volcano.kim
```

5. Testing

Command	What it does
`make test`	Runs the test programs in `tests/`, `tests/examples/` and `tests/errors/` with the interpreter and compares the output with `tests/expected/`.
`make test-compiler`	Compiles all the test programs, runs them, and checks they print the same thing as the interpreter.
`make test-chem`	Tests the chemistry functions in `src/chem.c` separately.

If everything works you should see:

```
make test           ----- 30 passed, 0 failed -----
make test-compiler  ----- 51 passed, 0 failed -----
```

6. The language

Variable and function names start with a small letter (`fuel`) and formulas start with a capital letter (`H2O`, `Ca(OH)2`). Comments are written with `//`. Simple statements end with `;`, and loops, conditions and functions use

`{ }` instead.

Statements

What	Kimiya
Assignment	`16 -> fuel;`
Output	`emit("Mass:", mass(H2O));`
If / else	`observe (x > 5) { ... } otherwise { ... }`
While loop	`heat while (temp < 100) { ... }`
Counting loop	`titrate g from 1 to 5 { ... }`
List loop	`each c in lab { ... }`
Function	`process add(a, b) { produce a + b; }`
Equation	`equation water: H2 + O2 -> H2O;`
Check	`check water;` or `check 2 H2 + O2 -> 2 H2O;`
List of formulas	`mixture lab = [H2O, CO2, NaCl];`
List of numbers	`readings trials = [5.1, 5.4, 4.2];`

Operators: `+ - * /` and `== != < > <= >=`.

Built-in functions

Call	Gives
`mass(F)`	molar mass of F
`moles(grams, F)`	grams converted to moles
`balance(eq)`	the balanced equation
`yield(eq, reactant, grams, product)`	grams of product made
`need(eq, reactant, grams, other)`	grams of the other reactant needed
`limiting(eq, A, gramsA, B, gramsB)`	the reactant that runs out first
`product(eq, A, gramsA, B, gramsB, P)`	grams of P made from both reactants
`count(list)`	number of items in a list
`average(readings)`	average of a readings list

Example

```
equation methane: CH4 + O2 -> CO2 + H2O;
balance(methane) -> methane;

16 -> fuel;
emit(yield(methane, CH4, fuel, CO2));    // grams of CO2 produced
emit(need(methane, CH4, fuel, O2));      // grams of oxygen needed

check CH4 + O2 -> CO2 + H2O;
```

Output:

```
43.89
63.82
CH4 + O2 -> CO2 + H2O is not balanced (H: left 4, right 2; O: left 2, right 3)
  balanced version: CH4 + 2 O2 -> CO2 + 2 H2O
```

Errors

When something is wrong the program stops and tells you the line, for example:

```
line 2: unknown element 'Xy' in formula Xy2O
line 2: undefined variable 'fuel'
line 4: N2 is not part of the equation CH4 + 2 O2 -> CO2 + 2 H2O
```

Changes from the proposal

1. `emit`, `observe` and `heat while` now need brackets, e.g. `emit("a", b);` and `observe (x > 5) {...}`.
2. `equation` doesn't balance the equation by itself anymore, it just stores it the way it's written. `check` tells you if it's balanced, and `balance()` balances it. Because of this the examples from the proposal need one extra line: `balance(name) -> name;`.
3. We added three extra functions: `balance`, `count` and `average`.

7. How it works

In Phase 1, Flex (`lexer.l`) breaks the program into tokens and Bison (`parser.y`) builds a syntax tree out of them. The interpreter, which is also in `parser.y`, then goes through the tree and runs it.

Phase 2 uses the same lexer, grammar and tree. The difference is that instead of running the tree, the compiler writes it out as LLVM IR. A Kimiya variable can hold a number at one point and a formula later, so we store every value in a small struct (we call it a box) that also remembers its type. These boxes and the functions that work on them (adding, comparing, printing etc.) are in `runtime.c`, and the IR just calls those functions. Conditions and loops are turned into basic blocks with `br`, and every `process` becomes a separate function in the IR.

8. Files

File	Description
`Makefile`	Builds the project and runs the tests.
`src/lexer.l`	The Flex file, splits the program into tokens.
`src/parser.y`	The Bison file. It has four parts: the syntax tree, the grammar, the interpreter, and the LLVM IR compiler. `main` is also here.
`src/chem.c`, `src/chem.h`	All the chemistry: the periodic table, reading formulas, molar mass, balancing, and the yield/limiting calculations.
`src/runtime.c`	The boxes and helper functions that compiled programs use (Phase 2).
`run_tests.sh`	Runs the Phase 1 tests (used by `make test`).
`kc.sh`	Compiles, links and runs one program (used by `make run`).
`tests/test2.kim` to `tests/test13.kim`	One test program for each feature.
`tests/examples/`	The five examples from our proposal.
`tests/errors/`	Programs that should give an error.
`tests/expected/`	Expected output of the tests, used by `make test`.
`tests/compile/`	Test programs for the compiler (Phase 2).
`tests/test_chem.c`	Tests for `chem.c`.